

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Отчет по лабораторным работам
по курсу «Информационный поиск»

Выполнил: Концебалов Олег Сергеевич

Группа: М8О-409Б-22

Преподаватель: Кухтичев Антон Алексеевич

Москва, 2025

Введение

Цель работы — разработать минимальный поисковый движок по собственному корпусу документов. Корпус собирается поисковым роботом (crawler) и хранится в MongoDB. Поиск реализован на C++ и включает токенизацию, нормализацию (стемминг), построение частотного распределения и проверку закона Ципфа, а также булев индекс и булев поиск. Для взаимодействия предусмотрены два интерфейса: CLI и веб-сервис (HTML-форма).

Анализ корпуса документов

1. Описание корпуса и характеристик документов

В качестве источников использованы русскоязычный сегмент Habr и Stack Overflow на русском языке. Документы представляют собой HTML-страницы (статьи, вопросы/ответы, обсуждения).

- Habr: статьи и страницы публикаций.
- Stack Overflow (ru): страницы вопросов и ответов.

Тип документа в корпусе: веб-страница новости/статьи в формате HTML

Язык: русский (основной контент), встречается код и вставки на английском

1.1. Из чего состоит «сырой» документ (HTML)

У обоих источников «сырой» документ (HTML) обычно содержит:

- основной контент статьи
- заголовок
- дата/время публикации
- рубрика/тема
- текст статьи (абзацы)
- изображения/видео (иногда) + подписи/источник фото
- ссылки внутри текста и блоки «по теме»
- навигация и сервисные блоки
- элементы авторизации/регистрации;
- комментарии (если есть) и формы обратной связи.
- канонический URL (canonical)

2. Выделение текста из «сырых» HTML документов

2.1. Что считать «текстом документа»

При индексации из HTML выделяется только содержательный текст:

- заголовок;
- основное тело статьи/вопроса (абзацы, списки);
- текст вопроса и ответа (для stackoverflow)

В проекте используется упрощённое «снятие» тегов: удаление HTML-тегов и спец-секций (script/style/noscript) с последующей нормализацией пробелов. Метод быстрый и простой, но может включать лишний текст (например, меню/комментарии) и хуже работает на сложной разметке. Для повышения качества можно заменить на DOM-парсер и CSS-селекторы для контейнера контента.

3. Проверка “пригодности корпуса”: существующий поиск по документам

Требование ЛР: если нельзя искать по документам существующими поисковиками — корпус использовать нельзя.

3.1. Встроенный поиск по сайту

- Habr: присутствует страница поиска <https://habr.com/ru/search/>
- Stackoverflow: присутствует страница поиска <https://ru.stackoverflow.com/search>

Вывод: корпус можно использовать — существует встроенный поиск у обоих источников.

3.2. Внешний поиск (Google / Яндекс) с ограничением на сайт

Можно искать по домену с оператором site: в Google. Также у Яндекса есть документные операторы для ограничения поиска по сайту/хосту/домену.

4. Примеры запросов к существующим поисковикам и недостатки выдачи

4.1. Примеры запросов (встроенный поиск)

Stackoverflow - запрос: c++

Habr - запрос: golang; Фильтры: по релевантности

4.2. Примеры запросов (Google/Яндекс с site:)

Google: site:ru.stackoverflow.com "c++" (кавычки для точной фразы)

Google: site:habr.com golang

Яндекс: site:habr.com "точная фраза" + слова темы

4.3. Недостатки поисковой выдачи

Дубли/переупаковки: одно и то же событие может быть опубликовано в нескольких версиях/обновлениях, и выдача показывает несколько почти одинаковых документов.

Зависимость от формулировки: без кавычек/точной фразы много “похожих” результатов, но не тех, которые нужны; с кавычками — наоборот, можно потерять релевантные документы из-за склонений/перефразирования.

Ранжирование “по свежести/популярности” (особенно на `habr`): вверху могут быть не самые “точные” совпадения, а самые свежие/кликабельные.

5. Статистика по корпусу

5.1. Общая статистика корпуса

Показатель	<code>habr.com</code>	<code>ru.stackoverflow.com</code>	Итого
Кол-во документов	30000	30000	60000
Raw объём, МВ	372.54	324.69	697.23
Выделенный текст, символов	222 994 921	197 750 214	420 745 135
Средний текст, символов/док	1627.80	1209.39	1418.59

6. Итоговый вывод

Корпус из материалов `habr.com` и `ru.stackoverflow.com` пригоден для выполнения последующих лабораторных работ, т.к.:

- документы имеют структурированный HTML с выделяемым основным текстом (абзацы/контейнеры статьи)
- существует проверяемый поиск по исходным документам: встроенный поиск обоих сайтов и внешний поиск с ограничением `site`

Поисковая система и Crawler

2.1. Общая схема работы

Поисковый робот обходит страницы целевых сайтов, извлекает ссылки на документы, скачивает HTML и сохраняет результат в MongoDB. Работа можно остановить и запустить снова — он продолжает обход с места остановки, используя коллекцию `frontier`.

2.2. Формат хранимого документа

В коллекции документов сохраняются поля:

- `url` — нормализованный URL документа;

- `raw_html` — «сырой» HTML документа;
- `source_name` — название источника (`habr` / `stackoverflow_ru`);
- `crawl_date` — дата обкачки (Unix time stamp).

2.3. Повторная обкачка и проверка изменений

При повторной обкачке документ обновляется только в случае изменений. Проверка может выполняться по хешу HTML, заголовкам ETag/Last-Modified или по сравнению нормализованного текста.

2.4. Конфигурация (YAML)

Робот получает единственный аргумент — путь к YAML-конфигу.

3. Индексация и поисковые компоненты (C++)

3.1. Источник данных для индексации

Индексатор читает документы напрямую из MongoDB, используя `mongo-cxxdriver`. Из каждого документа извлекаются `url`, `source_name`, `crawl_date` и `raw_html`.

3.2. Токенизация

Токенизация выполняется по следующим правилам:

- текст переводится в нижний регистр;
- токеном считается последовательность букв/цифр (включая кириллицу);
- знаки пунктуации и служебные символы выступают разделителями.

Недостатки: возможны «неудачные» токены (например, `c++17`, `e-mail`, `url`, `3.14`, слова с дефисами).

Улучшение: отдельные правила для дефисов, апострофов, сокращений, а также выделение токенов из `camelCase`.

3.3. Статистика токенизации и время работы

Индексатор выводит требуемые статистики: количество токенов, среднюю длину токена, а также время выполнения и скорость токенизации (KB/s).

Пример запуска в Docker:

```
docker compose run --rm search /app/indexer \
--mongo-uri "mongodb://ir-mongo:27017" \
--db "mai_ir_crawler" \
--collection "documents" \
--index "/data/index.bin" \
```

```
--zipf "/data/zipf.csv"
```

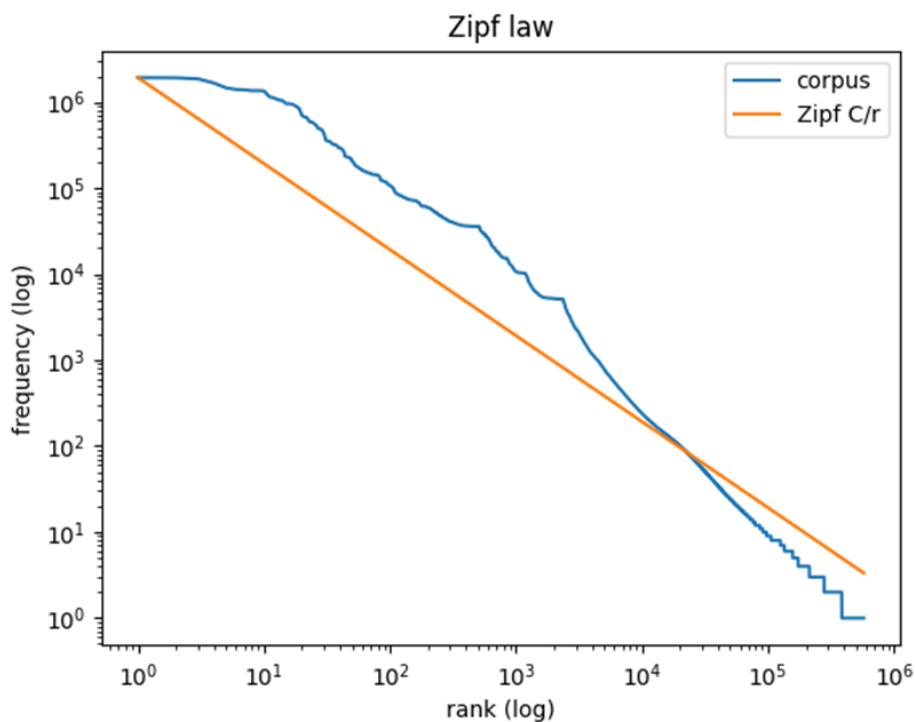
Вывод программы (полученные значения):

```
Docs: 60000
Terms: 2 273 547
Tokens: 1 094 348 149
Avg token length (codepoints): 7.16
Elapsed: 873 s
Tokenization speed: 12 843 KB/s
Saved index: /data/index.bin
Saved Zipf CSV: /data/zipf.csv
```

3.4. Закон Ципфа

После индексации строится распределение частот терминов по рангам.

Индексатор сохраняет CSV-файл (zipf.csv), содержащий rank, freq и значение аппроксимации Zipf C/r .



3.5. Лемматизация / стемминг

Для нормализации словоформ используется упрощённый стеммер (поддержка русского и английского). Нормализация применяется на этапе индексации и на этапе обработки запроса.

Оценка качества: сравнение выдачи до/после стемминга на нескольких запросах, включая случаи ухудшения (омонимия, чрезмерное обрезание суффиксов).

3.6. Булев индекс

Булев (инвертированный) индекс хранит для каждого термина список идентификаторов документов, в которых он встречается. Для словаря и списков документов используются собственные структуры данных (без `map/unordered_map`).

3.7. Булев поиск и интерфейсы

Поддерживаются операции AND, OR, NOT и круглые скобки. Результат поиска — список документов (`url` и источник). Реализованы два интерфейса:

- CLI: запросы читаются из `stdin`, результаты выводятся в `stdout`.
- Web: HTTP-сервер с HTML-формой ввода и HTML-выдачей (на базе `cpp-httpplib`).

4. Примеры запросов и проверка работы

4.1. CLI

Пример запуска CLI-поиска (индекс должен быть построен заранее):

```
docker compose run --rm search /app/search_cli --index /data/index.bin
# stdin:
docker AND kubernetes
```

4.2. Web

Пример запуска веб-сервиса:

```
docker compose up -d search
# go to http://localhost:8080 in browser
```

Примеры запросов для проверки:

- `docker AND compose`
- `kubernetes OR helm`
- `NOT windows`
- `(python OR rust) AND (parser OR html)`

IR Search

query	Search
------------------------------------	---------------------------------------

Syntax: **AND / OR / NOT**, parentheses. Implicit AND: `rust mongodb`

[← back](#)

Query: (python AND FastAPI) AND NOT shop

Hits: 161

1. <https://habr.com/ru/sandbox/> (habr.com) [\[view raw html\]](#)
2. <https://habr.com/ru/hubs/webdev/> (habr.com) [\[view raw html\]](#)
3. <https://habr.com/ru/companies/yandex/articles/> (habr.com) [\[view raw html\]](#)
4. <https://habr.com/ru/hubs/python/posts/> (habr.com) [\[view raw html\]](#)
5. <https://habr.com/ru/hubs/sci-fi/posts/> (habr.com) [\[view raw html\]](#)
6. <https://habr.com/ru/companies/otus/posts/> (habr.com) [\[view raw html\]](#)
7. https://habr.com/ru/hubs/it_testing/posts/ (habr.com) [\[view raw html\]](#)
8. <https://habr.com/ru/hubs/github/posts/> (habr.com) [\[view raw html\]](#)
9. <https://habr.com/ru/hubs/study/> (habr.com) [\[view raw html\]](#)
10. <https://habr.com/ru/hubs/robot/> (habr.com) [\[view raw html\]](#)
11. <https://habr.com/ru/companies/selectel/articles/975270/> (habr.com) [\[view raw html\]](#)
12. <https://habr.com/ru/companies/selectel/articles/975270/comments/> (habr.com) [\[view raw html\]](#)
13. <https://habr.com/ru/hubs/hackathons/> (habr.com) [\[view raw html\]](#)
14. <https://habr.com/ru/hubs/DIY/articles/page3/> (habr.com) [\[view raw html\]](#)
15. <https://habr.com/ru/hubs/easyelectronics/articles/page2/> (habr.com) [\[view raw html\]](#)
16. <https://habr.com/ru/companies/sberdevices/articles/> (habr.com) [\[view raw html\]](#)

Query: rust AND unsafe AND cloudflare

Hits: 5

1. https://habr.com/ru/hubs/network_hardware/posts/page2/ (habr.com) [\[view raw html\]](#)
2. <https://habr.com/ru/articles/969618/comments/> (habr.com) [\[view raw html\]](#)
3. <https://habr.com/ru/companies/domclick/articles/970010/comments/> (habr.com) [\[view raw html\]](#)
4. <https://habr.com/ru/news/977958/comments/> (habr.com) [\[view raw html\]](#)
5. <https://habr.com/ru/companies/selectel/articles/945534/comments/> (habr.com) [\[view raw html\]](#)

9. Итоговые выводы

В ходе выполнения данных лабораторных работ я познакомился с тем, как устроены современные поисковые системы, и даже сумел реализовать свою собственную.

Моя поисковая система:

- содержит расширяемый бинарный индекс, включающий обратный и прямой индексы, пригодный для булева поиска.
- реализует булев поиск с поддержкой AND, OR, NOT, пробелов и скобок;
- запускается в Docker
- предоставляет два интерфейса – CLI и Web
- предоставляет возможность смотреть на raw HTML выдаваемых документов

Скорость выполнения запросов находится в миллисекундном диапазоне для типовых случаев; длительная работа возникает на выражениях с большими объединениями и отрицаниями частых термов.

В целом поисковую систему можно дорабатывать и приделывать к ней новые функции, которые позволят улучшить качество и скорость поиска